C++20 Modules - Modern Code Organization

Theory, Edge Cases & Code Examples

- ✓ The Problem with Header Files
- ✓ The Solution
- ✓ Module Declaration and Export
- ✓ Module Partitions
- ✓ Module Implementation Units

→ Swipe to learn →

C++20 Modules - Modern Code Organization

Theory

📌 1. The Problem with Header Files (1/2)

Issues with Traditional Headers:

```
// ❌ Traditional header-based code

// math.h
#ifndef MATH_H
#define MATH_H

#include <iostream>  // Transitive inclusion
#include <vector>

class Math {
public:
    static int add(int a, int b);
    static int multiply(int a, int b);
};

#endif

// main.cpp
#include "math.h"  // Entire header parsed EVERY time
```

Problems:

1. **Slow compilation**: Headers parsed in every translation unit
2. **Order dependency**: `#include` order matters
3. **Macro pollution**: Macros leak across boundaries
4. **Fragile**: Include guards required, easy to break
5. **Transitive includes**: Unnecessary dependencies
6. **One Definition Rule violations**: Easy to violate accidentally

C++20 Modules - Modern Code Organization

Theory

📌 1. The Problem with Header Files (2/2)

Example of the Problem:

```
// header1.h
#define MAX 100

// header2.h
#include "header1.h"
// MAX is now defined here too! (macro pollution)

// main.cpp
#include "header2.h"
#undef MAX  // Required to avoid conflicts
```

C++20 Modules - Modern Code Organization

Theory

📌 2. C++20 Modules - The Solution

Benefits:

1. **Faster compilation**: Modules compiled once, reused
2. **Explicit exports**: Only exported names are visible
3. **No macro leakage**: Macros don't cross module boundaries
4. **Order-independent**: Import order doesn't matter
5. **Isolated**: Implementation details hidden by default
6. **Better tooling**: IDEs can understand module structure

Basic Module Example:

```
// math_module.cpp - Module Interface
export module math;  // Declare module named "math"

export int add(int a, int b) {
    return a + b;
}

export int multiply(int a, int b) {
    return a * b;
}

// main.cpp - Module User
import math;  // Import the math module

int main() {
    int result = add(5, 3);      // ✅ OK: add is exported
    int product = multiply(4, 2); // ✅ OK: multiply is exported
}
```

C++20 Modules - Modern Code Organization

Theory

📌 3. Module Declaration and Export

Module Interface Unit:

```
// my_module.cpp
export module my_module;  // Module declaration

// Export individual declarations
export int public_function() {
    return 42;
}

// Non-exported (internal linkage within module)
int internal_function() {
    return 100;
}

// Export multiple declarations at once
export {
    class MyClass {
    public:
        void method();
    };

    void another_function();
}
```

Using the Module:

```
// main.cpp
import my_module;

int main() {
    public_function();        // ✅ OK: exported
    internal_function();      // ❌ Error: not exported

    MyClass obj;              // ✅ OK: exported
    obj.method();
}
```

C++20 Modules - Modern Code Organization

Theory

📌 4. Module Partitions

Purpose: Split large modules into smaller logical parts.

Primary Module Interface:

```
// math.cpp
export module math;

export import :arithmetic;  // Re-export arithmetic partition
export import :geometry;    // Re-export geometry partition

// Optionally add more exports here
export int compute_something();
```

Module Partitions:

```
// math-arithmetic.cpp
export module math:arithmetic;  // Partition declaration

export int add(int a, int b) { return a + b; }
export int subtract(int a, int b) { return a - b; }

// math-geometry.cpp
export module math:geometry;

export double area_circle(double radius) { return 3.14159 * radius * radius; }
export double area_square(double side) { return side * side; }
```

Using Partitioned Module:

```
// main.cpp
import math;  // Imports primary module interface (includes partitions)

int main() {
    add(5, 3);              // From :arithmetic partition
    area_circle(10.0);      // From :geometry partition
    compute_something();    // From primary module
}
```

C++20 Modules - Modern Code Organization

Theory

📌 5. Module Implementation Units

Separate Interface from Implementation:

```
// math.ixx (Module Interface)
export module math;

export class Calculator {
public:
    int add(int a, int b);
    int multiply(int a, int b);
private:
    int internal_state;
};

// math.cpp (Module Implementation)
module math;  // Implementation unit (no export keyword)

int Calculator::add(int a, int b) {
    return a + b;
}

int Calculator::multiply(int a, int b) {
    return a * b;
}
```

C++20 Modules - Modern Code Organization

Theory

📌 6. Importing Standard Library Modules

Traditional vs Module Import:

```
// ❌ Traditional headers
#include <iostream>
#include <vector>
#include <string>

// ✅ C++23: Standard library modules (future)
import std;         // All standard library
import std.io;      // Just I/O
import std.containers;  // Just containers
```

C++20 Reality:

Most compilers don't fully support standard library modules yet. Use:

```
// C++20: Mix of headers and modules
#include <iostream>  // Still need headers for std library
import my_custom_module;
```

C++20 Modules - Modern Code Organization

Theory

📌 7. Global Module Fragment

Purpose: Include traditional headers for backward compatibility.

```
// my_module.cpp
module;  // Global module fragment

// Traditional headers go here
#include <iostream>
#include <vector>

export module my_module;

export void print_vector(const std::vector<int>& vec) {
    for (int val : vec) {
        std::cout << val << ' ';
    }
}
```

Why Needed:

- Bridges gap between headers and modules
- Allows gradual migration
- Required for headers that can't be modules yet

C++20 Modules - Modern Code Organization

Theory

📌 8. Private Module Fragment

Purpose: Hide implementation details completely.

```
// my_module.cpp
export module my_module;

export class MyClass {
public:
    void public_method();
};

module :private;  // Private module fragment

// Implementation details hidden here
void MyClass::public_method() {
    // Implementation
}

// Helper functions (not visible to importers)
static void internal_helper() {
    // ...
}
```

C++20 Modules - Modern Code Organization

Theory

📌 9. Module Linkage and Visibility

Module Linkage Rules:

```
export module example;

// Module linkage (internal to module, but usable across TUs in this module)
int module_internal = 42;

// Exported (module linkage, visible to importers)
export int exported_var = 100;

// Internal linkage (like static)
namespace {
    int truly_internal = 10;
}

// External linkage (global, visible outside module)
extern "C" int c_compatible_function() {
    return 5;
}
```

C++20 Modules - Modern Code Organization

Theory

📌 10. Migration Strategy: Headers to Modules

Step 1: Create Module Interface

```
// Old: math.h
#ifndef MATH_H
#define MATH_H
int add(int a, int b);
#endif

// New: math.ixx
export module math;
export int add(int a, int b);
```

Step 2: Provide Header Compatibility

```
// math_compat.h - For users still using headers
#include "math_traditional.h"

// math_traditional.h
#ifndef MATH_TRADITIONAL_H
#define MATH_TRADITIONAL_H
int add(int a, int b);
#endif
```

Step 3: Gradual Migration

- New code uses `import math;`
- Legacy code uses `#include "math_compat.h"`
- Eventually remove compatibility headers

C++20 Modules - Modern Code Organization

Edge Cases

📌 1. Macros Don't Cross Module Boundaries

```
// module1.cpp
export module module1;

#define MY_MACRO 42
export int get_value() { return MY_MACRO; }

// main.cpp
import module1;

int x = MY_MACRO;  // ❌ Error: MY_MACRO not defined
// Macros are NOT exported!
```

Solution: Use constexpr instead of macros in modules.

C++20 Modules - Modern Code Organization

Edge Cases

📌 2. Import Order Doesn't Matter (But Helps)

```
// Both are equivalent
import module_a;
import module_b;

import module_b;
import module_a;
```

But module build order matters for the compiler!

C++20 Modules - Modern Code Organization

Edge Cases

📌 3. One Definition Rule Still Applies

```
// module1.cpp
export module module1;
export struct Data { int x; };

// module2.cpp
export module module2;
export struct Data { int x; };  // Different module, same name

// main.cpp
import module1;
import module2;

Data d;  // ❌ Ambiguous: Which Data?
```

C++20 Modules - Modern Code Organization

Edge Cases

📌 4. Template Instantiation Visibility

```
// module.cpp
export module templates;

template<typename T>
struct Container {  // Not exported
    T value;
};

export template<typename T>
void process(Container<T> c) {  // Uses non-exported template
    // ...
}

// main.cpp
import templates;

Container<int> c;  // ❌ Error: Container not exported
process(c);        // ❌ Error: Can't instantiate
```

Solution: Export templates that need to be instantiated by users.

C++20 Modules - Modern Code Organization

Code Examples

📌 1. Simple Utility Module (1/2)

```
// utils.ixx - Module Interface
export module utils;

#include <string>
#include <vector>

export namespace utils {
    std::string to_upper(const std::string& str);
    std::vector<std::string> split(const std::string& str, char delim);
}

// utils.cpp - Module Implementation
module utils;

#include <algorithm>
#include <sstream>

std::string utils::to_upper(const std::string& str) {
    std::string result = str;
    std::transform(result.begin(), result.end(), result.begin(), ::toupper);
    return result;
}

std::vector<std::string> utils::split(const std::string& str, char delim) {
    std::vector<std::string> tokens;
    std::stringstream ss(str);
    std::string token;
    while (std::getline(ss, token, delim)) {
        tokens.push_back(token);
    }
    return tokens;
}

// main.cpp - Usage
import utils;
#include <iostream>

int main() {
    std::cout << utils::to_upper("hello") << '\n';  // "HELLO"
// ...continues on next slide
```

C++20 Modules - Modern Code Organization

Code Examples

📌 1. Simple Utility Module (2/2)

[Code continues across slides]

```
// ...continued from previous slide

    auto tokens = utils::split("a,b,c", ',');
    for (const auto& token : tokens) {
        std::cout << token << ' ';  // "a b c"
    }
}
```

C++20 Modules - Modern Code Organization

Code Examples

📌 2. Module with Partitions (1/2)

```
// graphics.ixx - Primary Interface
export module graphics;

export import :shapes;
export import :colors;

export void render_scene();

// graphics-shapes.ixx - Shapes Partition
export module graphics:shapes;

export struct Point { double x, y; };

export class Circle {
    Point center;
    double radius;
public:
    Circle(Point c, double r);
    double area() const;
};

// graphics-colors.ixx - Colors Partition
export module graphics:colors;

export struct Color {
    uint8_t r, g, b;
};

export constexpr Color RED{255, 0, 0};
export constexpr Color GREEN{0, 255, 0};
export constexpr Color BLUE{0, 0, 255};

// graphics.cpp - Implementation
module graphics;

void render_scene() {
    Circle c{{0, 0}, 5.0};
    // Use colors and shapes
}
// ...continues on next slide
```

C++20 Modules - Modern Code Organization

Code Examples

📌 2. Module with Partitions (2/2)

[Code continues across slides]

```
// ...continued from previous slide

// main.cpp - Usage
import graphics;

int main() {
    Circle c{{10, 20}, 15};
    Color col = RED;
    render_scene();
}
```

C++20 Modules - Modern Code Organization

Code Examples

📌 3. Module with Private Implementation (1/2)

```
// database.ixx
export module database;

#include <string>
#include <memory>

// Forward declaration
class ConnectionImpl;

export class Database {
public:
    Database(const std::string& connection_string);
    ~Database();

    void execute(const std::string& query);

private:
    std::unique_ptr<ConnectionImpl> impl_;
};

module :private;  // Private fragment

#include <iostream>

// Implementation details hidden
class ConnectionImpl {
public:
    void connect(const std::string& conn_str) {
        std::cout << "Connecting to " << conn_str << '\n';
    }

    void run_query(const std::string& query) {
        std::cout << "Executing: " << query << '\n';
    }
};

Database::Database(const std::string& connection_string)
    : impl_(std::make_unique<ConnectionImpl>()) {
    impl_->connect(connection_string);
// ...continues on next slide
```

C++20 Modules - Modern Code Organization

Code Examples

📌 3. Module with Private Implementation (2/2)

[Code continues across slides]

```
// ...continued from previous slide
}

Database::~Database() = default;

void Database::execute(const std::string& query) {
    impl_->run_query(query);
}

// main.cpp
import database;

int main() {
    Database db("localhost:5432");
    db.execute("SELECT * FROM users");
}
```

C++20 Modules - Modern Code Organization

Code Examples

📌 4. Header Compatibility Layer

```
// math_module.ixx - Module Interface
export module math;

export int add(int a, int b) { return a + b; }
export int subtract(int a, int b) { return a - b; }

// math.h - Header Compatibility (for legacy code)
#ifndef MATH_H
#define MATH_H

#ifdef __cpp_modules
// If modules supported, just import
import math;
#else
// Otherwise, provide traditional declarations
int add(int a, int b);
int subtract(int a, int b);
#endif

#endif

// math_impl.cpp - Implementation for header users
#ifndef __cpp_modules
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
#endif

// modern_code.cpp - Uses modules
import math;

// legacy_code.cpp - Uses headers
#include "math.h"
```

C++20 Modules - Modern Code Organization

Code Examples

📌 5. Template Module (1/2)

```
// container.ixx
export module container;

export template<typename T>
class MyVector {
    T* data_;
    size_t size_;
    size_t capacity_;

public:
    MyVector() : data_(nullptr), size_(0), capacity_(0) {}

    void push_back(const T& value) {
        if (size_ == capacity_) {
            reserve(capacity_ == 0 ? 1 : capacity_ * 2);
        }
        data_[size_++] = value;
    }

    T& operator[](size_t index) { return data_[index]; }
    size_t size() const { return size_; }

private:
    void reserve(size_t new_capacity);
};

// Implementation must be in interface for templates
template<typename T>
void MyVector<T>::reserve(size_t new_capacity) {
    T* new_data = new T[new_capacity];
    for (size_t i = 0; i < size_; ++i) {
        new_data[i] = data_[i];
    }
    delete[] data_;
    data_ = new_data;
    capacity_ = new_capacity;
}

// main.cpp
// ...continues on next slide
```

C++20 Modules - Modern Code Organization

Code Examples

📌 5. Template Module (2/2)

[Code continues across slides]

```
// ...continued from previous slide
import container;

int main() {
    MyVector<int> vec;
    vec.push_back(1);
    vec.push_back(2);
    vec.push_back(3);
}
```

What You Learned

Key Takeaways

- ✓ The Problem with Header Files
- ✓ The Solution
- ✓ Module Declaration and Export
- ✓ Module Partitions

→ Swipe to learn →